

RESUME PROJECT PLAN · 2026

Growmart **ERP** Software

A full-stack multi-tenant accounting & inventory management system with GST compliance, RBAC, and real-time financial reporting

14

Weeks

6

Phases

11

Voucher Types

17

Reports

Client: Growmart / V Dev Associates Management Consultancy · Stack: Angular 21 · NestJS · PostgreSQL ·
Type: ERP / SaaS



Is this a good resume project?

Honest evaluation of scope, complexity, and employer appeal

✓ Strong Points

- **Double-entry accounting engine** — extremely rare among junior devs
- **Multi-tenant architecture** with complete data isolation per company
- **4-level RBAC** (Super Admin → Admin → Accountant → Billing Staff)
- **FIFO inventory costing** — used in real warehouses
- **GST compliance** (India-specific, niche, real-world demand)
- **17 financial reports** including P&L, Balance Sheet, Trial Balance
- **Audit log** — voucher log + user activity log
- Matches Tally ERP in feature depth — shows enterprise-grade thinking

⚠ Risks to Manage

- Full scope is 4–6 months of solo work — plan a **MVP milestone** at Week 8
- Looks like "a Tally clone" if not framed correctly — emphasize **what you built**, not what it resembles
- GST is India-specific; frame global-facing features (RBAC, multi-tenancy) prominently for non-India employers
- Accounting domain knowledge required — invest 1 week learning double-entry basics before coding
- No novel ML / WebRTC angle — but real-world complexity compensates

Verdict: YES — build it. This project demonstrates the exact depth that separates junior developers from mid-level ones: complex business logic, multi-tenant data isolation, RBAC, a financial calculation engine, and audit trails. Even at 60% completion (through Phase 4), it is more impressive than most portfolio projects. Frame it as *"a GST-compliant multi-company ERP with a from-scratch double-entry accounting engine"* — not as "a Tally clone."



Recommended Tech Stack

Chosen to match your existing Angular skills while adding backend depth

FRONTEND

Angular 21

Your current stack. Standalone components + Signals. Angular CDK for drag/barcode.

BACKEND

NestJS (Node.js)

TypeScript-first, decorator-based, mirrors Angular patterns. Easy to learn alongside Angular.

DATABASE

PostgreSQL + Prisma

Row-level tenant isolation via company_id. Prisma ORM for type-safe queries.

AUTH

JWT + bcrypt

Stateless auth. Role stored in token. Per-company session with companyId claim.

EXPORT

ExcelJS + xml2js

Excel (.xlsx) and XML export for all 17 reports as per spec.

DEPLOY

Docker + Railway / Render

Free tier hosting. Docker compose for local dev (API + PostgreSQL).

Why NestJS over Express? NestJS gives you modules, guards (RBAC), interceptors (audit logging), and pipes (validation) out of the box — each maps directly to a Growmart requirement. The learning curve is low if you already know Angular since both share decorator-based architecture.



6-Phase, 14-Week Delivery Plan

Each phase ends with a testable, working deliverable

Phase	Focus	Weeks	Key Deliverable	Status
Phase 1	Project Foundation & Auth	1-2	Login working, company creation, user RBAC enforced	Not Started
Phase 2	Master Data Module	3-4	All masters (Accounting, Inventory, GST) with permission guards	Not Started
Phase 3	Voucher Engine	5-8	All 11 voucher types with double-entry posting & GST auto-split	Not Started
Phase 4	Reports Module	9-11	P&L, Balance Sheet, Trial Balance, Stock & GST reports	Not Started
Phase 5	Advanced Features	12-13	Excel/XML export, bill-wise aging, barcode scanner, audit logs	Not Started
Phase 6	Polish & Deploy	14	Docker deploy, demo data, README, portfolio writeup	Not Started

MVP Checkpoint at Week 8: After Phase 3, you have a working accounting + inventory system with voucher entry. This alone is portfolio-ready. Phase 4–6 add depth.



14-Week Gantt Chart

Visual timeline across all phases

[illegible]

1 Foundation, Auth & Company Management

Weeks 1–2

Repo setup → database → JWT auth → multi-company RBAC → UI shell

Day 1–2: Project scaffold — Angular 21 (frontend) + NestJS (backend) monorepo. Docker Compose with PostgreSQL. ESLint + Prettier. Git repository with branching strategy (main / dev / feature/*).

Day 3–4: Database schema — Prisma schema for Company, User, UserRole, AuditLog. Every table includes company_id for tenant isolation. Migration baseline.

Day 5–6: Authentication — NestJS JWT auth module. bcrypt password hashing. Login endpoint returns token with {userId, companyId, role} claims. Angular AuthGuard + RoleGuard reads token.

Day 7–8: Super Admin dashboard — Company creation form (Name, Address, GST Number, GST State Code, Email, Mobile, Books Beginning Date). List companies, switch active company. Super Admin bypasses company_id filter in all queries.

Day 9–10: User management — CRUD for users within a company. Assign roles (Admin / Accountant / Billing Staff). Backdated-entry window config (Accountant = 2 days). Permission middleware as NestJS Guards reusable across all future modules.

Day 11–14: Angular shell — Main layout (sidebar + header). Route guards per role. Company selector in header. "Books Beginning Date" validation service (blocks all entries before this date globally). Toast notification system.

2 Master Data Module

Weeks 3–4

Accounting Group → Ledger → Unit → GST Rate → Inventory Group → Inventory Item

Step 1: Accounting Group Master — Seed 4 default groups (I/E/A/L) as system-protected records. UI to create up to 4 sub-layers beneath each. Unique name validation. Opening balance fields (debit/credit) only for Asset and Liability groups. Permission guard: Level-3 create-only, Level-1/2 full CRUD.

Step 2: Accounting Ledger Master — Auto-numbering (group code + sequence, e.g. E1, E2). Fields: Name, Group, State, GST Number, Address, Mobile. Soft-delete guard (block delete if ledger used in any voucher). Seed 15 predefined system ledgers (Purchase, Sales-Shop, Cash, GPay, CGST, SGST, etc.) as system-reserved (no rename/delete).

Step 3: Unit Master — Formal Name, Symbol, Decimal Places. Examples: Number (Nos, 1), Kilogram (Kg, 3). Dropdown reused in Inventory Item Master.

Step 4: GST Rate Master — Enter IGST rate, auto-compute CGST = SGST = IGST/2. Store all three. Used in Inventory Item and Purchase/Sales vouchers.

Step 5: Inventory Group Master — Seed "Primary" as default. Single-layer sub-groups with unique names. Permission guard same as Accounting Group.

Step 6: Inventory Item Master — Fields: Name, Inventory Group, Unit, GST Rate, Opening Quantity, Purchase Price per Unit, Total Value. Barcode field (optional ISBN/EAN for scanner integration in Phase 5). FIFO lot tracking starts from opening entry.

Week 4 Checkpoint: Demo login as Super Admin → create a company → log in as that company's Admin → create ledgers and inventory items. All master forms should be working with role-based field locking before moving to vouchers.

3 Voucher Engine — All 11 Voucher Types

Double-entry posting · GST auto-split · FIFO costing · Bill-wise settlement

Weeks 5–8

Week 5 — Core Posting Engine — Build the `VoucherPostingService`: a single backend function that takes a voucher and writes atomic debit/credit pairs to a `LedgerEntry` table. Every voucher flows through this service. Ensures Books Beginning Date validation, running balance update, auto voucher numbering (per company, per voucher type, resettable per financial year).

Week 5 — Purchase Voucher — Fields: Date, Vendor Ledger, Ref No., Item lines (Item + Units + Unit Price + GST inclusive/exclusive toggle + Discount). Auto-compute: GST split (CGST/SGST or IGST based on state), Net Payable, Old Balance Payable, Total Payable. Bill Hold/Recall (save as Draft). Posting: Debit Purchase, Debit CGST/SGST, Credit Vendor Ledger.

Week 6 — Sales Shop & Online Vouchers — Same structure as Purchase but reversed. Multi-mode payment split (Cash / GPay / PhonePe / BharatPe / Card / Bank). If payment received → auto-generate Receipt voucher posting for each mode. Customer mandatory for Online sales. Discount auto-posts to "Discount Given" ledger.

Week 6 — Sales Return & Purchase Return — Linked to original voucher via reference number. Reverses stock (adds back) and ledger entries. Partial return support (return fewer units than original).

Week 7 — Scrap Voucher & Physical Stock — Scrap: FIFO weighted-average price lookup per item → debit "Loss due to Scrap" expense ledger, credit stock. Physical Stock: compare Book Stock vs Physical Count → auto-post difference as stock loss with adjustment voucher. Closing Stock = Book Stock – Missing Stock value.

Week 7 — Payment & Receipt Vouchers — Multi-mode payment (split across Cash, GPay, PhonePe, Card, Bank ledgers). Bill-wise settlement: show pending bills for vendor/customer → allow partial/full settlement → update outstanding balance. Mode of payment auto-selects corresponding bank/cash ledger.

Week 8 — Contra & Journal Vouchers — Contra: only Cash/Bank group ledgers. From/To ledger selection. Fund transfer with debit on From, credit on To. Journal: any ledger, standard debit/credit pair, reference field. FROM and TO entry UI.

Week 8 — Voucher List & Rights — Voucher list view per type with date range filter. Inline edit (same-day) and cancel (with checker authentication for Billing Staff). Role-based cancel/modify controls. Checker auth flow (another Billing Staff or higher must approve). MVP milestone demo.

**Accounting Logic Quick Reference**

Double-entry rules your posting engine must implement

Voucher	Debit	Credit	Stock Effect
Purchase	Purchase A/c + CGST/SGST	Vendor (Payable)	Stock IN (FIFO lot added)
Sales	Customer / Cash / Bank	Sales A/c + CGST/SGST	Stock OUT (FIFO lot consumed)
Sales Return	Sales A/c (reversal)	Customer	Stock IN (returned)
Purchase Return	Vendor	Purchase A/c (reversal)	Stock OUT (returned)
Payment	Vendor (reduces payable)	Cash / Bank	No change
Receipt	Cash / Bank	Customer (reduces receivable)	No change
Contra	To-Account (Cash/Bank)	From-Account (Cash/Bank)	No change
Scrap	Loss due to Scrap (Expense)	Stock A/c	Stock OUT (FIFO weighted avg)
Physical Stock	Stock Loss (Expense)	Stock A/c	Stock adjusted to physical qty

4 Reports Module — 17 Financial Reports

Weeks 9–11

All reports: date filter · drill-down to voucher · edit if authorized

Week 9 — Core Financial Statements

P&L Statement: From–To date range. Group incomes and expenses. Compute Gross Profit (Sales – COGS) and Net Profit (Gross – Indirect Expenses).

Balance Sheet: As-on-date snapshot. Assets (Current + Fixed + Stock) = Liabilities (Current + Capital) balanced.

Trial Balance: All ledgers with opening + period debit/credit + closing balance. Debit total = Credit total validation.

Week 9 — Ledger & Day Book

General Ledger: All entries for a selected ledger with running balance. Date range filter.

Day Book: All vouchers in chronological order. Grouped by date. Each row clickable → opens voucher detail.

Week 10 — Receivable & Payable Reports — Outstanding balance per party. Bill-wise details (which voucher created the outstanding). Aging buckets: 0–30 days / 31–60 days / 61–90 days / 90+ days. As-on-date query (time-travel: show what was outstanding on any past date).

Week 10 — GST Filing Report — As per GST portal format. GSTR-1 equivalent (sales invoices grouped by party + HSN). GSTR-2B equivalent (purchase invoices). Tax summary (CGST/SGST/IGST collected vs paid). HSN-wise summary table.

Week 11 — Stock Reports

Stock Summary: All items with opening qty, inward, outward, closing qty + value. As-on-date or period.

Stock Movement Register: Per item: Opening → all IN/OUT movements → Closing. Drill-down to voucher.

Item Movement Register: Single item detailed movement.

Item Price List: Purchase price, sales price, average price per item.

Week 11 — Analysis Reports

Item-wise Analysis: For each item — from whom purchased (avg purchase price) and to whom sold (avg sales price) → profit per item.

Party-wise Analysis: For each vendor/customer — items transacted, cost vs sales price, profit.

Ledger List & Voucher Log: All ledgers in system. Full voucher history with modification timestamps.

5 Advanced Features

Weeks 12–13

Export · Barcode · Bill Hold · Audit Log · Backdated Entry Approval

Excel & XML Export — All 17 reports exportable via ExcelJS (.xlsx with formatted headers, color coding) and xml2js (.xml for ERP upload). Single "Export" button per report with format dropdown.

Barcode Scanner Integration — USB HID barcode scanner emulates keyboard input. Angular service listens for rapid keystrokes ending in Enter → treats as barcode. Auto-fills item in voucher line without mouse click.

Bill Hold / Recall — Save in-progress voucher as Draft with a name/tag. Recall drafts from list in voucher screen. Auto-clear drafts older than 24h.

Audit & User Logs — Every create/update/delete writes to AuditLog table (user, timestamp, action, entity, old value, new value). User Log: login/logout/report-export events. Viewable by Admin/Super Admin. Exportable.

Backdated Entry Approval Workflow — Accountant submits backdated entry → enters Pending Approval state. Admin sees approval queue → approve or reject with note. Approved entries post to ledger. Rejected entries returned with comment.

Checker Authentication Flow — Billing Staff actions (create master, same-day modify) require a second Billing Staff or higher to authenticate (re-enter their password in a modal). Implemented as a reusable Angular dialog + NestJS decorator.

6 Polish, Deploy & Portfolio Writeup

Docker · Demo data · Live URL · Resume bullets

Week 14

Docker Compose — Single docker-compose.yml running Angular (nginx), NestJS, PostgreSQL. One-command setup for local demo. Add .env.example.

Deploy to Railway / Render — Free tier. NestJS API + PostgreSQL. Angular build deployed to Vercel/Netlify. Live demo URL you can share in your resume.

Demo Data Seed — Realistic seed: 2 companies, all user levels, 50+ inventory items, 200+ vouchers, populated reports. Interviewers should be able to click around without setup.

Portfolio README — Architecture diagram (Mermaid), feature list with screenshots, setup instructions, "what I learned" section for interview talking points.



Resume Bullets — Copy-Ready

How to frame this project for maximum impact

RESUME PROJECT ENTRY

Built a full-stack multi-tenant ERP (Angular 21 + NestJS + PostgreSQL) with a from-scratch double-entry accounting engine, FIFO inventory costing, and GST (CGST/SGST/IGST) auto-computation across 11 voucher types.

Implemented 4-level RBAC (Super Admin / Admin / Accountant / Billing Staff) with company data isolation, checker authentication workflow, and backdated-entry approval queue.

Delivered 17 financial reports (P&L, Balance Sheet, Trial Balance, GST Filing, Receivable Aging) with Excel/XML export, date-range filtering, and role-gated drill-down to individual voucher entries.

Integrated USB barcode scanner via HID keyboard emulation, multi-mode payment splitting (Cash/GPay/UPI/Bank), and comprehensive audit logging for all user and voucher actions.



Interview Talking Points

What to say when an interviewer asks "tell me about this project"

On the accounting engine: "The hardest part was building the double-entry posting engine — every voucher must create balanced debit/credit pairs atomically. I modelled it as a VoucherPostingService that takes a voucher schema and writes to a normalized LedgerEntry table. All reports then aggregate from that table, which means the Balance Sheet and P&L always balance by construction."

On multi-tenancy: "Every database table has a company_id column. The NestJS auth guard injects the company context from the JWT token and a Prisma middleware automatically appends WHERE company_id = ? to every query. Super Admin bypasses this filter, giving them cross-company visibility."

On FIFO: "Inventory uses FIFO lot tracking — each purchase creates a 'lot' with price and quantity. Sales consume lots in order of purchase date. The scrap voucher uses the weighted average price across current lots to value the scrapped goods."

On RBAC complexity: "The spec has 4 user levels with different permissions for create, modify, delete, and even a 'checker' authentication flow where a Billing Staff action needs a second user to approve. I implemented this as composable NestJS Guards so I could mix-and-match permission requirements per endpoint."

Final tip: Push to GitHub with a clear README and a live demo link. When you add it to your resume, link the GitHub repo. Even if only Phases 1–4 are complete, the accounting engine + RBAC + vouchers + reports puts this ahead of 95% of portfolio projects at the junior-to-mid level.



Prisma Schema Outline

Core tables — every table (except Company, SuperAdmin) has company_id for tenant isolation

Company id Int PK code String(5) unique name String gstNumber String gstStateCode String booksBeginDate DateTime address String	User id Int PK company_id FK → Company name String email String unique passwordHash String role Enum (L1-L4) backdateDays Int default 0	AccountingGroup id Int PK company_id FK code String (I/E/A/L) name String unique parentId FK self (depths4) reportingHead Enum PL/BS normalBalance Enum DR/CR isSystem Boolean
Ledger id Int PK company_id FK code String (E1, A3...) name String groupId FK → AccGroup gstNumber String? openingBalance Decimal isSystem Boolean	Voucher id Int PK company_id FK type Enum (11 types) voucherNo String auto-gen refNo String? date DateTime status Enum Draft/Posted/Cancelled partyLedgerId FK → Ledger	LedgerEntry id Int PK company_id FK voucherId FK → Voucher ledgerId FK → Ledger date DateTime debit Decimal credit Decimal
InventoryItem id Int PK company_id FK name String groupId FK → InvGroup unitId FK → Unit gstRateId FK → GSTRate openingQty Decimal barcode String?	StockLot (FIFO) id Int PK company_id FK itemId FK → InvItem voucherId FK → Voucher date DateTime qty Decimal unitPrice Decimal qtyRemaining Decimal	AuditLog id Int PK company_id FK userId FK → User action Enum CREATE/UPDATE/DELETE entity String (table name) entityId Int oldValue Json? newValue Json?

Key design decision: The LedgerEntry table is the single source of truth for all balances. P&L, Balance Sheet, Trial Balance, and Ledger reports all read from this table using $\text{SUM}(\text{debit}) - \text{SUM}(\text{credit})$ aggregations grouped by ledger and date range. This ensures consistency — there is no separate "balance" column that can drift out of sync.

